



PROJE ANALİZ RAPORU

Ders: Yazılım Mühendisliği
Bölüm/Şube: Bilgisayar Mühendisliği
Grup Adı: Grup 2
Proje Adı: Banka Yönetim Sistemi
Teslim Tarihi: 22.03.2026

Grup Üyeleri:

- Abdullah Arpacı 23100011033
- Yemliha Can Yastı 24100011016
- Duran Can Çetin 24100011024
- Yaşar Samet Üçoğlu 24100011032
- Muhammed Yusuf Erzi 24100011052

İçindekiler

1. Giriş.....	3
2. Genel Bakış	3
3. Önerilen Sistem.....	3
3.1 İşlevsel Gereksinimler (Functional Requirements).....	4
3.2 İşlevsel Olmayan Gereksinimler (Non-Functional Requirements)	4
3.3 Sözde Gereksinimler (Pseudo Requirements)	5
4. Sistem Modelleri	5
4.1 KULLANIM DURUMU (Use Case) MODELİ.....	6
4.2 Sınıf Diyagramı	15
4.3 Durum Diyagramı.....	16
4.4 Aktivite Diyagramı	17
4.5 Sıra (Sequence) Diyagramı	19
4.6 Kullanıcı Arayüzü.....	23
5. Referanslar	23

1. Giriş

Projenin temel amacı;C++ dilinin sunduğu yüksek performans ve nesne yönelimli programlama prensiplerini kullanarak güvenli, modüler ve kullanıcı dostu bir finans yönetim platformu oluşturmaktır. Projenin kapsamı; kullanıcı doğrulama sistemleri, hesap yönetimi ve temel finansal işlemler gibi operasyonel süreçlerin masaüstü ortamında simüle edilmesini, güçlü bir yerel veri işleme mekanizmasını ve etkileşimli bir grafik kullanıcı arayüzünü içermektedir. Bu raporun yapısı ise projenin önerilen sistem vizyonunu, hedef kitlesini ve yazılımın yerine getirmesi gereken işlevsel, performans ve tasarım kısıtlarını içeren detaylı analizleri adım adım takip edecek şekilde kurgulanmıştır.

2. Genel Bakış

Projenin Amacı: Nesne yönelimli mimari (OOP) ve modern bir grafik arayüz (GUI) aracılığıyla, temel bankacılık süreçlerini dijital ortamda simüle eden eğitim odaklı bir yazılım geliştirmektir.

Hedef Kullanıcılar: * Yöneticiler (Admin): Müşteri hesaplarını oluşturan, güncelleyen ve sistem güvenliğini sağlayan yetkili personeller.

Müşteriler: Kendi sanal bakiyelerini yöneten, hesap hareketlerini izleyen ve işlem yapan bireysel kullanıcılar.

Temel İşlevler: Sanal bakiye üzerinden para yatırma, çekme ve havale/virman işlemlerinin gerçekleştirilmesi; kullanıcı verilerinin şifrelenmesi ve tüm sistem hareketlerinin zaman damgasıyla loglanması.

Kapsam ve Sınırlar: C++ ve Qt kullanılarak geliştirilen bu masaüstü uygulaması; gerçek finansal ağlara (SWIFT, POS), dış API'lere veya fiziksel donanımlara (ATM) entegre olmadan, tamamen kapalı devre bir yerel veritabanı üzerinde (SQLite/MySQL) çalışacak şekilde sınırlandırılmıştır.

3. Önerilen Sistem

C++ programlama dilinin sunduğu yüksek performans, nesne yönelimli tasarım ve verimli bellek yönetimi avantajlarını kullanarak geliştirilecek, profesyonel standartlarda bir masaüstü bankacılık platformudur. Sistem, hem bireysel bankacılık hizmetlerinden yararlanmak isteyen kullanıcılar hem de bankanın yetkili personeli için ortak bir çalışma alanı sağlar. Tasarlanacak proje; hesap açılışından para transferlerine geniş bir fonksiyon yelpazesini, verilerin korunduğu güvenli bir altyapı ile sunar.

3.1 İşlevsel Gereksinimler (Functional Requirements)

Bankacılık Sistemi, kullanıcı yönetimi, hesap işlemleri ve güvenlik olmak üzere üç temel işlevsel alan üzerine kurulmuştur.

Kullanıcı Yönetimi

Sistem iki farklı kullanıcı rolünü destekler. Yönetici (Admin); yeni müşteri kaydı oluşturma, bilgi güncelleme ve hesap dondurma/silme yetkilerine sahiptir. Müşteri ise kendi bakiyesini ve hesap hareketlerini görüntüleyebilir.

Hesap ve Para İşlemleri

Kullanıcılar hesap açma/kapatma, bakiye sorgulama, para yatırma, para çekme ve hesaplar arası havale yapabilir. Tüm bakiye güncellemeleri eşzamanlı gerçekleşir. Sistem aynı zamanda TL ile döviz arasında temel alım-satım işlemlerini de destekler.

Güvenlik ve Loglama

Sisteme erişim kullanıcı adı ve şifre doğrulamasıyla sağlanır; hassas veriler temel şifreleme mekanizmalarıyla korunur. Başarılı/başarısız girişler ve para transferleri dahil tüm işlemler, tarih-saat damgasıyla log dosyasına kaydedilir.

3.2 İşlevsel Olmayan Gereksinimler (Non-Functional Requirements)

Bu bölüm, sistemin "ne yapacağından" ziyade "nasıl bir kalitede" çalışacağını belirler:

- **Güvenlik:** Sisteme erişim kullanıcı adı ve şifre doğrulamasıyla sağlanmalı , hassas veriler (bakiye, şifre vb.) temel şifreleme mekanizmalarıyla korunarak saklanmalıdır.
- **Veri Bütünlüğü (ACID):** Finansal işlemler sırasında veri kaybını önlemek için işlemler Atomicity, Consistency, Isolation ve Durability (ACID) prensiplerine tam uyumlu gerçekleştirilmelidir.
- **İzlenebilirlik (Loglama):** Başarılı veya başarısız tüm giriş denemeleri ve para transferleri; işlem tipi, tarih ve saat damgasıyla birlikte bir log dosyasında kayıt altına alınmalıdır.
- **Performans:** Para yatırma, çekme ve havale gibi bakiye güncellemeleri sistem üzerinde eşzamanlı ve anlık olarak gerçekleşmelidir.
- **Kullanılabilirlik (UX):** Arayüz tasarımı; öğrenciler, yaşlılar ve beyaz yaka çalışanlar dahil tüm kullanıcı kitlelerinin zorlanmadan işlem yapabileceği, modern ve kullanıcı dostu bir yapıda olmalıdır.

- **Hata Yönetimi:** Uygulama çalışma anında oluşabilecek kritik hataları yakalamalı ve kullanıcıyı doğru şekilde bilgilendirmelidir

3.3 Sözde Gereksinimler (Pseudo Requirements)

Bu bölüm, projenin geliştirilmesi aşamasında uymak zorunda olduğunuz teknik kısıtları içerir:

- **Programlama Dili:** Yazılımın tamamı yüksek performans ve nesne yönelimli programlama (OOP) desteği sunan **C++** dili ile geliştirilmelidir.
- **Arayüz Framework'ü:** Grafik kullanıcı arayüzü (GUI) tasarımı için **Qt framework**'ü kullanılmalıdır.
- **Platform:** Uygulama yalnızca **Masaüstü (Desktop)** platformunda çalışacak şekilde tasarlanmalı; mobil veya web sürümü geliştirilmemelidir.
- **Veri Saklama:** Verilerin kalıcılığını sağlamak amacıyla yerel bir veritabanı (**SQLite/MySQL**) veya yüksek performanslı dosya tabanlı bir kayıt sistemi kullanılmalıdır.
- **Mimari Yapı:** Proje, yazılım mühendisliği standartlarına uygun olarak nesne yönelimli (OOP) ve katmanlı mimari prensiplerine göre kurgulanmalıdır.
- **Dış Entegrasyon Kısıtı:** Sistem tamamen kapalı devre bir simülasyon olarak çalışmalı; dış API'lere (döviz kuru vb.) veya gerçek bankacılık ağlarına (SWIFT, POS) bağlanmamalıdır.

4. Sistem Modelleri

Bu bölüm, bankacılık sisteminin teknik mimarisini ve işleyişini UML diyagramları aracılığıyla analiz eder. Sistem modelleri, yazılımın nesne yönelimli yapısını, kullanıcı etkileşimlerini ve operasyonel iş akışlarını görselleştirerek projenin tasarım mantığını somutlaştırır.

Analiz kapsamında kullanılan **Kullanım Durumu, Sınıf, Sıra ve Aktivite** diyagramları; sistemin modülerliğini, veri bütünlüğünü ve güvenliğini doğrulamak amacıyla kurgulanmıştır. Aşağıdaki alt başlıklarda, bu modellerin detayları ve senaryo bazlı açıklamaları yer almaktadır.

4.1 KULLANIM DURUMU (Use Case) MODELİ

Sistemin dış aktörlerle (Müşteri, Yönetici) olan etkileşimlerini görselleştiren bu model, yazılımın sunduğu temel fonksiyonları kullanıcı bakış açısıyla tanımlar. Her bir kullanım durumu, sistemin hangi senaryoda nasıl bir değer ürettiğini gösterir.

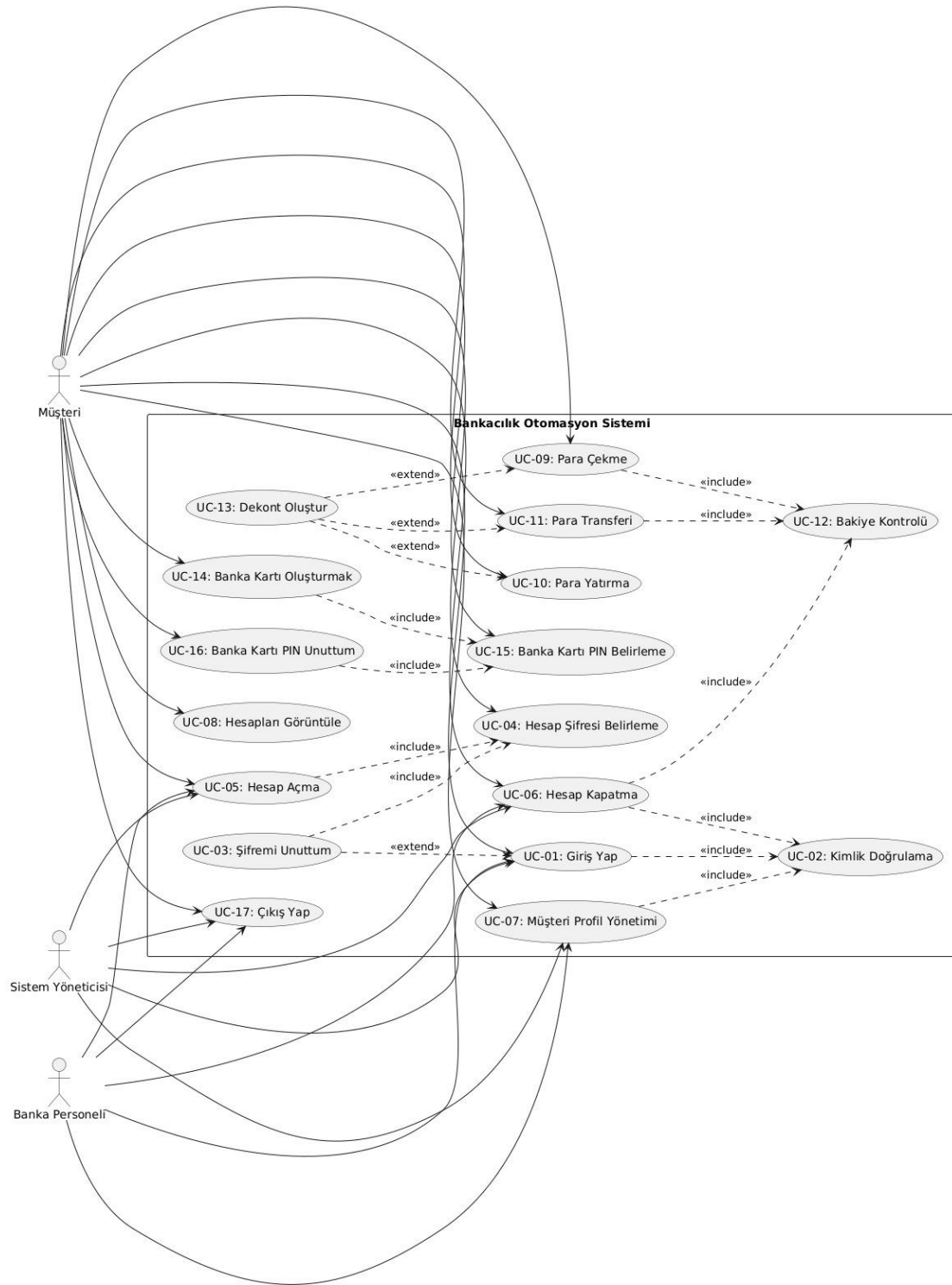


Figure 1:Use Case Modeli

UC-01: Giriş Yap

- Aktörler: Müşteri, Banka Personeli, Sistem Yöneticisi
- Giriş Koşulu: Kullanıcının sistemde kayıtlı bir hesabı olmalıdır.
- Çıkış Koşulu:
 - Oturum başlatılır.
 - Kullanıcı ana ekrana yönlendirilir.
- Olay Akışı:
 - Kullanıcı, sistemin giriş arayüzüne kimlik numarasını ve şifresini girer.
 - Sistem, arka planda zorunlu olarak Kimlik Doğrulama (UC-02) işlemini çalıştırır.
 - Doğrulama başarılıysa sistem ana ekranı açar.
 - Eğer kullanıcı şifresini hatalı girerse veya talep ederse Şifremi Unuttum (UC-03) sürecine geçiş yapabilir.

UC-02: Kimlik Doğrulama

- Aktörler: Sistem
- Giriş Koşulu: Kullanıcı arayüzden giriş verilerini iletmış veya sistem içinde bir işlem tetiklenmiş olmalıdır.
- Çıkış Koşulu:
 - Doğrulama başarılı veya başarısız sonucu döner.
 - Asıl işlem bu sonuca göre onaylanır veya reddedilir.
- Olay Akışı:
 - Sistem, girilen kimlik ve şifre verilerini kayıtlı veriler ile karşılaştırır.
 - Eşleşme durumunu kontrol eder.
 - Sonucu, bu işlemi çağıran kullanım durumuna iletir.

UC-03: Şifremi Unuttum

- Aktörler: Müşteri, Banka Personeli, Sistem Yöneticisi
- Giriş Koşulu:
 - Kullanıcı giriş ekranında olmalıdır.
 - Sistemde halihazırda kayıtlı bir hesabı bulunmalıdır.
- Çıkış Koşulu:
 - Kullanıcının eski şifresi geçersiz kılınır.
 - Yeni şifre kullanıma alınır.
- Olay Akışı:
 - Kullanıcı giriş arayüzünde Şifremi Unuttum bağlantısına tıklar.
 - Müşteriye güvenlik ve ek doğrulama soruları sorulur.
 - Doğrulama başarıyla geçildiğinde sistem, yeni şifre belirlenmesi için zorunlu olarak Hesap Şifresi Belirleme (UC-04) arayüzünü açar.

UC-04: Hesap Şifresi Belirleme

- Aktörler: Müşteri
- Giriş Koşulu: Müşteri, hesap açma (UC-05) işlemini başlatmış veya şifre sıfırlama (UC-03) adımından bu ekrana yönlendirilmiş olmalıdır.
- Çıkış Koşulu: Belirlenen yeni şifre sisteme kaydedilir.
- Olay Akışı:
 - Sistem, kullanıcıdan güvenlik uygun yeni bir şifre girmesini ister.
 - Sistem kayıtları günceller.

UC-05: Hesap Açma

- Aktörler: Müşteri, Banka Personeli, Sistem Yöneticisi
- Giriş Koşulu:
 - Gerekli kişisel veriler eksiksiz girilmiş olmalıdır.
 - Mevcut müşteri yeni hesap açıyorsa sisteme giriş yapmış olmalıdır.
- Çıkış Koşulu: Sistemde yeni bir hesap oluşturulur.

- Olay Akışı:
 - Aktör, yeni hesap açma formunu doldurur ve onaylar.
 - Sistem bilgilerin doğruluğunu kontrol eder ve hesap kaydını oluşturur.
 - Kayıt oluştuktan sonra sistem, hesabın kullanılabilmesi için müşteriye zorunlu olarak Hesap Şifresi Belirleme (UC-04) ekranına yönlendirir.

UC-06: Hesap Kapatma

- Aktörler: Müşteri, Banka Personeli, Sistem Yöneticisi
- Giriş Koşulu:
 - Sisteme giriş yapılmış olmalıdır.
 - Kapatılacak hesabın bakiyesi sıfır olmalıdır.
- Çıkış Koşulu: Sistem kayıtlarından hesap silinir ve kullanılamaz hale getirilir.
- Olay Akışı:
 - Aktör kapatılacak hesabı seçer.
 - Sistem zorunlu olarak Kimlik Doğrulama (UC-02) adımını çağırır.
 - Doğrulama sonrası sistem arka planda zorunlu olarak Bakiye Kontrolü (UC-12) işlemini çalıştırır.
 - Şartlar sağlanıyorsa hesap kalıcı olarak silinir ve kullanıcıya bilgi verilir.

UC-07: Müşteri Profil Yönetimi

- Aktörler: Müşteri, Banka Personeli, Sistem Yöneticisi
- Giriş Koşulu:
 - Sisteme giriş yapılmış olmalıdır.
 - Aktif bir oturum açılmış olmalıdır.
- Çıkış Koşulu: Sistemdeki müşteri verileri güncellenir.
- Olay Akışı:

- Aktör Profil Ayarları arayüzüne girerek güncellenecek yeni bilgilerini form üzerine yazar.
- Sistem, değişiklikleri onaylamadan önce güvenlik için zorunlu olarak Kimlik Doğrulama (UC-02) işlemi talep eder.
- Doğrulama başarılıysa yeni veriler sisteme yazılır.

UC-08: Hesapları Görüntüle

- Aktörler: Müşteri
- Giriş Koşulu: Sisteme başarılı bir şekilde giriş yapılmış olmalıdır.
- Çıkış Koşulu: Müşterinin tüm güncel hesap durumları, bilgileri ve anlık bakiyeleri arayüzde listelenir.
- Olay Akışı:
 - Müşteri menüden Hesaplarım sekmesine tıklar.
 - Sistem üzerinden ilgili müşteriye ait tüm hesap verileri sorgulanır.
 - Gelen veriler formatlanarak kullanıcı ekranında gösterilir.

UC-09: Para Çekme

- Aktörler: Müşteri
- Giriş Koşulu: Sisteme giriş yapılmış olmalıdır.
- Çıkış Koşulu: Seçilen hesabın bakiyesi, çekilen tutar kadar eksilir.
- Olay Akışı:
 - Müşteri para çekmek istediği hesabı ve çekilecek tutarı seçer.
 - Sistem, işlem yapılmadan önce zorunlu olarak Bakiye Kontrolü (UC-12) işlemine girer.
 - Bakiye yeterliyse tutar hesaptan düşülür.
 - İşlem başarıyla bittiğinde müşteri isterse Dekont Oluştur (UC-13) süreci tetiklenir.

UC-10: Para Yatırma

- Aktörler: Müşteri
- Giriş Koşulu:

- Sisteme giriş yapılmış olmalıdır.
 - Paranın ekleneceği geçerli bir hesap seçilmelidir.
- Çıkış Koşulu: Seçilen hesabın bakiyesi, yatırılan nakit tutarı kadar artar.
- Olay Akışı:
 - Müşteri yatırılacak tutarı ve hedef hesabı sisteme bildirir.
 - Sistem paranın girişini onaylar ve bakiyeyi günceller.
 - İşlem başarıyla bittiğinde müşteri isterse Dekont Oluştur (UC-13) süreci tetiklenir.

UC-11: Para Transferi

- Aktörler: Müşteri
- Giriş Koşulu:
 - Sisteme giriş yapılmış olmalıdır.
 - Hedef alınan alıcı IBAN/Hesap numarasının geçerli bir formatta olması gerekir.
- Çıkış Koşulu:
 - Gönderici hesaptan ilgili tutar düşer.
 - Alıcı hesaba eklenir.
- Olay Akışı:
 - Müşteri transfer edilecek tutarı, paranın çekileceği kendi hesabını ve hedef alıcı hesabını arayüze girer.
 - Sistem, transfer öncesi zorunlu olarak Bakiye Kontrolü (UC-12) işlemini çalıştırır.
 - Bakiye yeterliyse transfer gerçekleştirilir.
 - İşlem başarıyla bittiğinde müşteri isterse Dekont Oluştur (UC-13) süreci tetiklenir.

UC-12: Bakiye Kontrolü

- Aktörler: Sistem
- Giriş Koşulu: Sistem içerisinde para çıkışı gerektiren bir işlem tetiklenmiş olmalıdır.

- Çıkış Koşulu: İşleme onay verilir veya yetersiz bakiye durumunda işlem durdurulur.
- Olay Akışı:
 - Sistem, kullanıcı tarafından talep edilen işlem tutarını, ilgili hesabın mevcut bakiyesiyle karşılaştırır.
 - İşlem tutarı mevcut bakiyeden küçük veya eşitse sürece onay verir.
 - Tutar bakiyeden büyükse asıl işlemi durdurur ve ekrana Yetersiz Bakiye hatasını yansıtır.

UC-13: Dekont Oluştur

- Aktörler: Müşteri
- Giriş Koşulu:
 - Sisteme giriş yapılmış olmalıdır.
 - Finansal bir işlem başarıyla sonuçlanmış olmalıdır.
- Çıkış Koşulu: İşlemin detaylarını barındıran dijital bir belge oluşturulur.
- Olay Akışı:
 - Finansal işlem başarıyla bittikten sonra müşteri ekrandaki Dekont Oluştur butonuna tıklar.
 - Sistem gerçekleşen işleme ait verileri formatlayarak bir şablon üzerine yerleştirir.
 - Oluşturulan belge müşteriye gösterilir.

UC-14: Banka Kartı Oluşturmak

- Aktörler: Müşteri
- Giriş Koşulu:
 - Sisteme giriş yapılmış olmalıdır.
 - Müşterinin kartı bağlayabileceği bir hesabı bulunmalıdır.
- Çıkış Koşulu: Yeni bir banka kartı numarası üretilerek sisteme işlenir.
- Olay Akışı:
 - Müşteri hesabını seçerek arayüz üzerinden Banka Kartı Oluştur talebi gönderir.

- Sistem talebi onaylar ve kart bilgilerini kaydeder.
- Kartın kullanılabilmesi için sistem zorunlu olarak Banka Kartı PIN Belirleme (UC-15) arayüzünü açar.

UC-15: Banka Kartı PIN Belirleme

- Aktörler: Müşteri
- Giriş Koşulu: Müşteri, Banka Kartı Oluşturmak (UC-14) veya PIN Unuttum (UC-16) süreçleri tarafından bu ekrana otomatik olarak yönlendirilmiş olmalıdır.
- Çıkış Koşulu: Girilen PIN sisteme kaydedilir.
- Olay Akışı:
 - Sistem müşteriden 4 haneli PIN girmesini talep eder.
 - Sistem PIN ve kartı eşleştirerek kaydeder.

UC-16: Banka Kartı PIN Unuttum

- Aktörler: Müşteri
- Giriş Koşulu:
 - Sisteme giriş yapılmış olmalıdır.
 - Müşterinin sistemde kayıtlı bir banka kartı bulunmalıdır.
- Çıkış Koşulu: Kartın eski 4 haneli PIN kodu geçersiz kılınır.
- Olay Akışı:
 - Müşteri ilgili kartı seçip PIN Unuttum seçeneğine tıklar.
 - Müşteriye güvenlik ve ek doğrulama soruları sorulur.
 - Doğrulama başarılı olduğu an sistem, müşteriye zorunlu olarak Banka Kartı PIN Belirleme (UC-15) arayüzüne yönlendirir.

UC-17: Çıkış Yap

- Aktörler: Müşteri, Banka Personeli, Sistem Yöneticisi
- Giriş Koşulu: Kullanıcının sistemde bir oturumu bulunmalıdır.
- Çıkış Koşulu: Kullanıcı giriş ekranına düşürülür.
- Olay Akışı:

- Kullanıcı arayüzdeki Çıkış butonuna tıklar.
- Sistem kullanıcıyı giriş ekranına yönlendirir.

4.2 Sınıf Diyagramı

Sınıf diyagramı, geliştirilecek bankacılık sisteminin nesne yönelimli programlama mimarisini, veri kapsülleme prensiplerini ve sistem bileşenleri arasındaki yapısal ilişkileri ortaya koymaktadır. C++ mimarisine uygun olarak tasarlanan bu yapı, kodun modülerliğini ve tekrar kullanılabilirliğini artırmayı hedeflemektedir

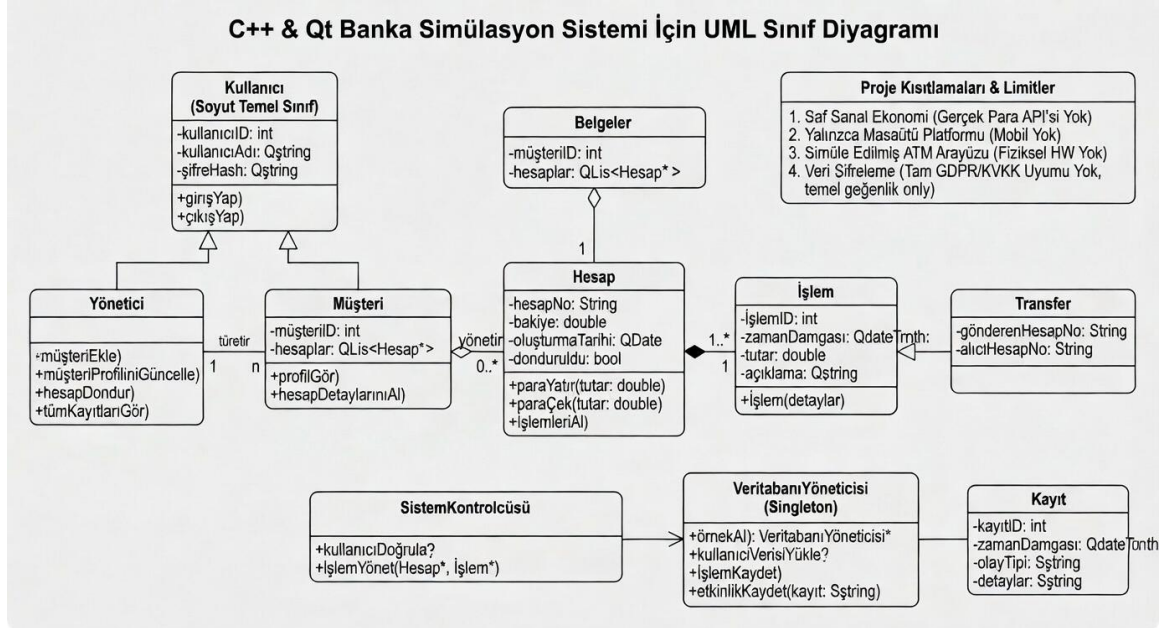


Figure 2: Bankacılık Sistemi Sınıf Diyagramı

Sistem mimarisini oluşturan temel sınıflar ve üstlendikleri görevler aşağıda özetlenmiştir:

- **Kullanici:** Sisteme erişim sağlayan tüm bireylerin (Müşteri veya Admin) ortak özelliklerini (Kimlik No, Ad, Soyad, Şifre) barındıran temel (Base) sınıftır. Sisteme güvenli girişi sağlayan girişYap() gibi ortak metotları içerir.
- **Müşteri:** Kullanici sınıfından kalıtım (inheritance) yoluyla türetilmiş alt sınıftır. Banka müşterilerinin kendi profillerini yönetebilmesi, yeni hesap açma/kapatma taleplerinde bulunabilmesi ve kendi hesapları arasında işlem yapabilmesi için gerekli fonksiyonları kapsar.
- **Admin:** Yine Kullanici sınıfından türetilen ve sistemdeki en yüksek yetkilere sahip olan sınıftır. Müşteri kayıtlarını onaylama, şüpheli durumlarda hesapları dondurma ve sistem genelindeki log (işlem) kayıtlarını inceleme görevlerini üstlenir.

- **Hesap:** Finansal işlemlerin merkezinde yer alan sınıftır. Müşteriye ait bakiye bilgisini kapsülleme (encapsulation) prensibiyle güvenli bir şekilde gizli tutar. paraYatir(), paraCek(), transferYap() ve bakiyeSorgula() gibi temel bankacılık operasyonlarını yürütür.
- **BankaKart :** Müşterilerin hesaplarına bağlı fiziksel veya sanal kart bilgilerini (Kart No, CVV, Son Kullanma Tarihi ve PIN) yöneten sınıftır. PIN doğrulama ve kart blokaj işlemlerini kontrol eder.
- **Islem / Log:** Sistemde gerçekleşen her türlü finansal hareketi (para transferi, çekim) ve kritik güvenlik olayını (başarılı/başarısız girişler) tarih, saat ve tutar bilgisiyle birlikte veritabanına yazmak üzere tasarlanmış kayıt sınıfıdır.

4.3 Durum Diyagramı

Bu durum diyagramı, sistemdeki bir "Banka Hesabı" nesnesinin yaratıldığı andan sistemden silindiği ana kadar geçirdiği evreleri (durumları) ve bu evreler arası geçiş koşullarını modellemektedir.

4.3 Banka Hesabı ve Kullanıcı Giriş Durum Diyagramı

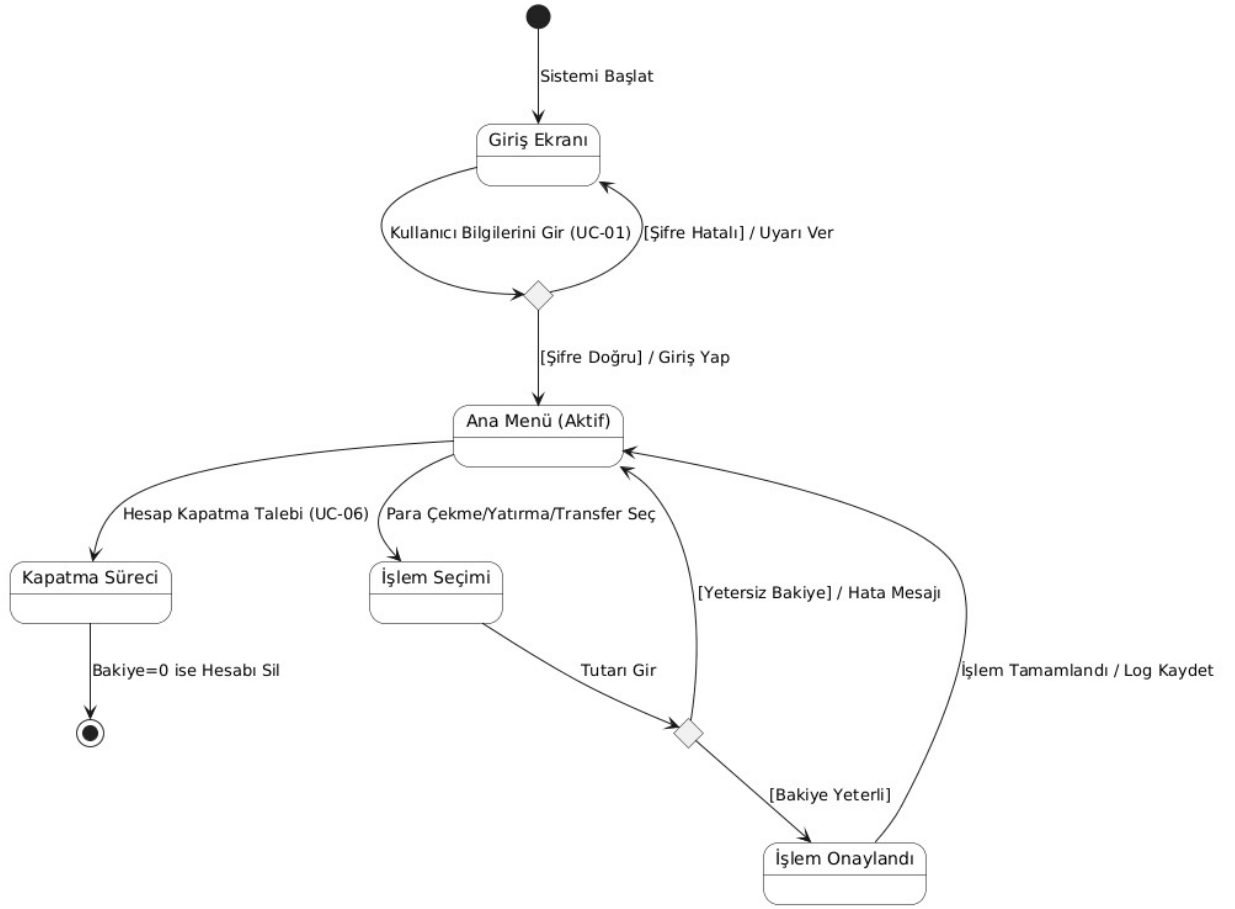


Figure 3: Banka Hesabı Yaşam Döngüsü Durum (State) Diyagramı

4.4 Aktivite Diyagramı

Aktivite diyagramları, sistemdeki iş süreçlerinin başlangıç noktasından bitişine kadar izlediği yolu, karar noktalarını ve paralel ilerleyen işlemleri görselleştirmek için kullanılır. Bu modelleme sayesinde, karmaşık bankacılık prosedürlerinin mantıksal akışı ve hata durumlarında sistemin nasıl tepki vereceği net bir şekilde ortaya konulmaktadır.

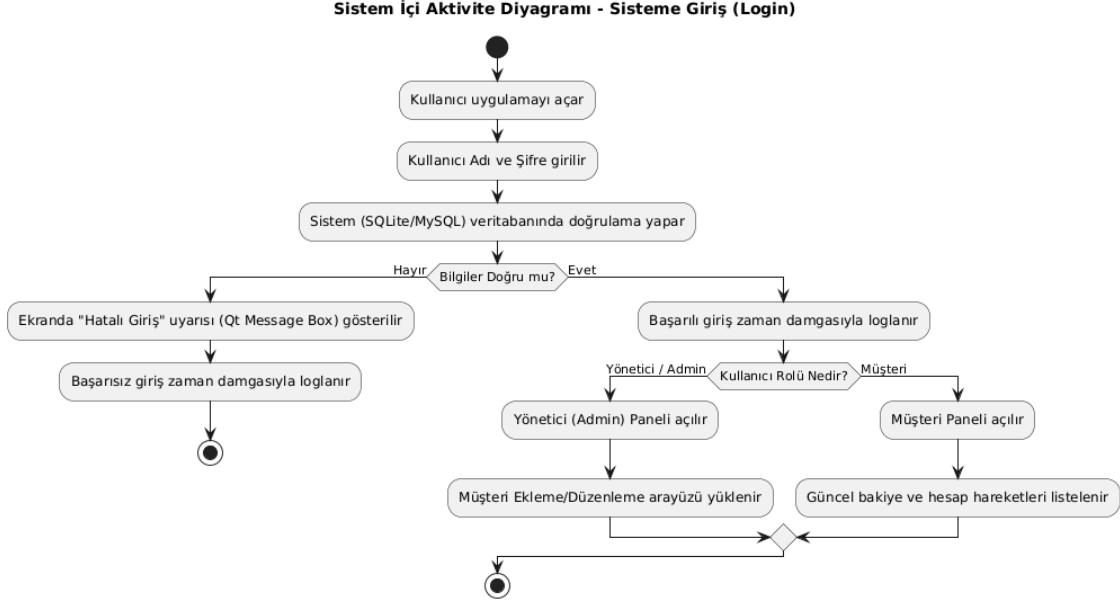


Figure 4: Sisteme Giriş (Login) ve Yetkilendirme Aktivite Diyagramı.

Bu aktivite diyagramı, kullanıcıların sisteme giriş sürecindeki mantıksal akışı ve karar düğümlerini modeller. Kullanıcı arayüzden kimlik bilgilerini girdiğinde, arka planda yerel veritabanı (SQLite/MySQL) sorgusu çalışır. Bilgilerin hatalı olması durumunda sistem başarısız işlemi loglayarak süreci sonlandırır. Doğrulama başarılı olduğunda ise, sistem kullanıcının yetki rolünü kontrol eder ve işlemi güvenli bir şekilde logladıktan sonra kullanıcıyı yetkisine uygun olan Yönetici

(Admin) veya Müşteri paneline yönlendirir.

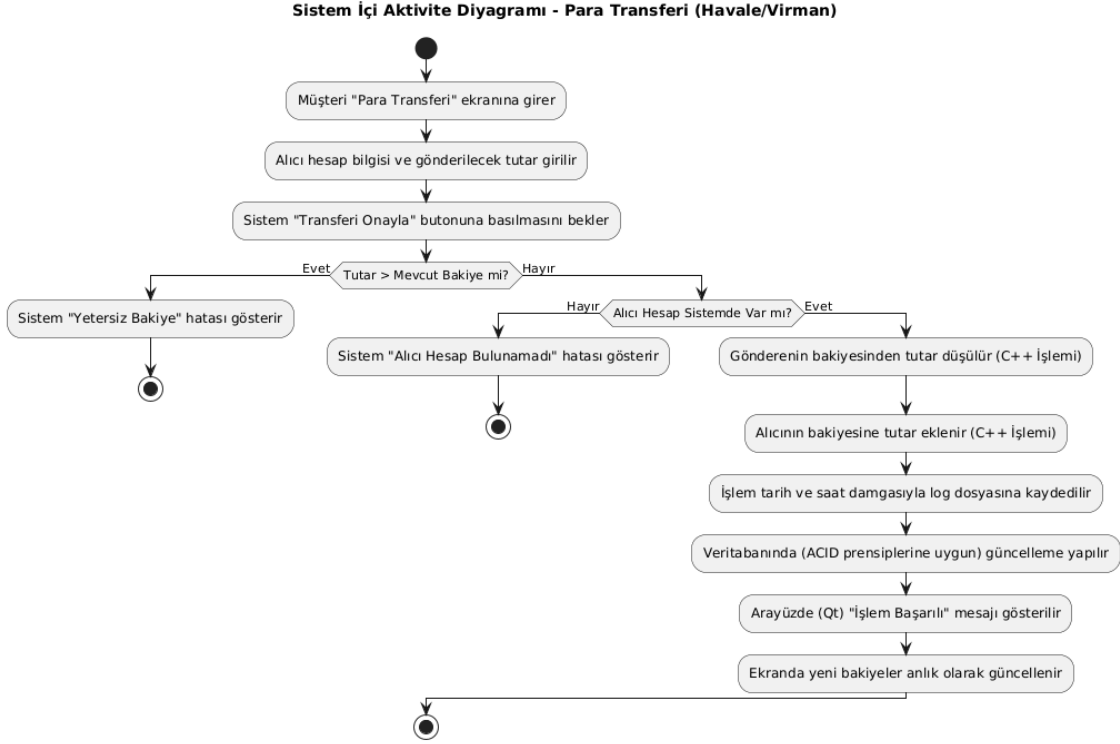


Figure 5:Para Transferi (Havale/Virman) İşlem Süreci Aktivite Diyagramı.

Bu diyagram, finansal bir işlem olan para transferinin sistem içindeki adım adım işleyişini göstermektedir. İşlem başlatıldığında sistem sırasıyla iki kritik kontrol (karar düğümü) yapar: Gönderenin bakiyesi yeterli mi ve alıcı hesap sistemde mevcut mu? Her iki şart sağlandığında, C++ arka plan algoritmaları devreye girerek bakiyeleri günceller. Veri bütünlüğünü sağlamak adına (ACID prensipleri doğrultusunda) veritabanı güncellemeleri eşzamanlı yapılır, işlem zaman damgasıyla loglanır ve son olarak Qt arayüzü üzerinden kullanıcıya işlemin başarılı olduğuna dair anlık geri bildirim sunulur.

4.5 Sıra (Sequence) Diyagramı

Bu bölümde, bankacılık sistemindeki nesnelerin belirli senaryolar dahilinde birbirleriyle olan etkileşimleri ve mesajlaşma sıraları incelenmektedir. Sıra diyagramları, sistemin çalışma anındaki dinamik davranışını ve verinin kullanıcı arayüzü, kontrol katmanı ve veritabanı arasındaki akışını zaman ekseninde görselleştirir.

Senaryo 1: Kullanıcının Sisteme Giriş Yapması ve Kimlik Doğrulaması

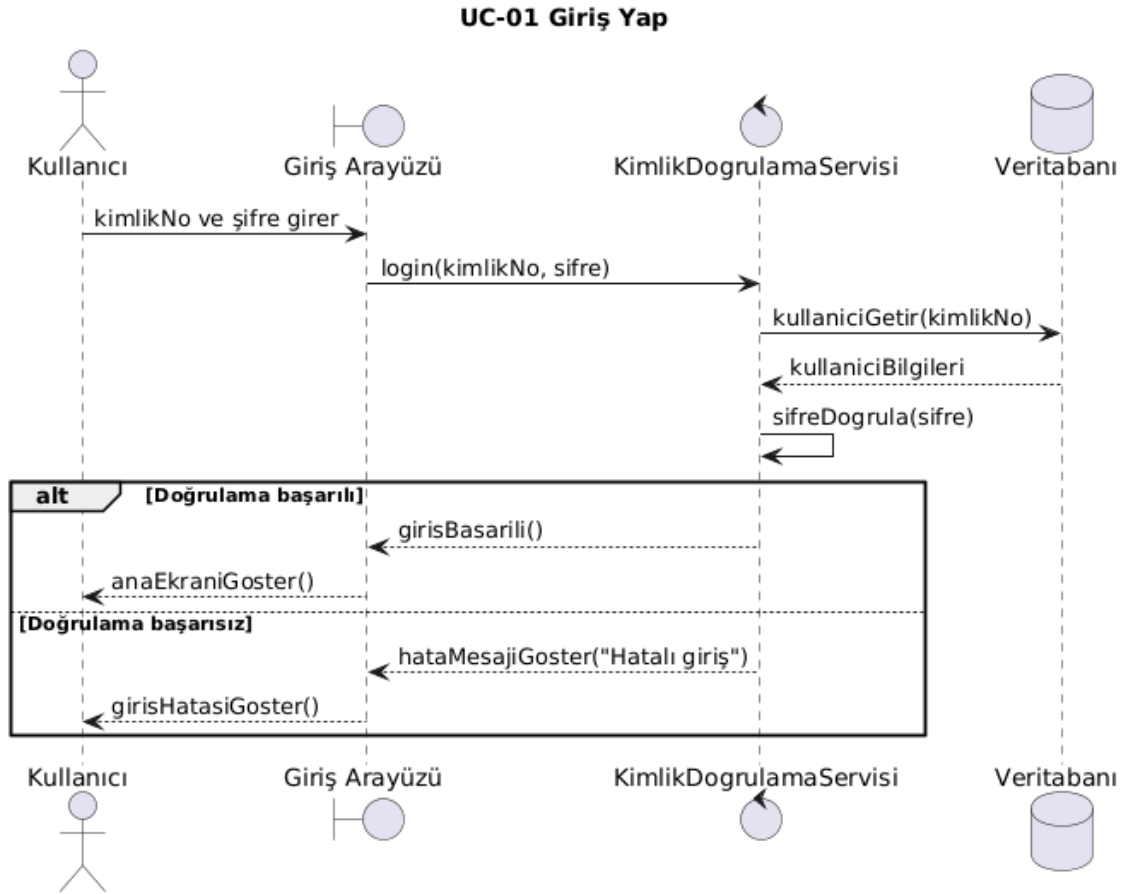


Figure 6:Kullanıcı Giriş İşlemi (UC-01) Sıra Diyagramı

Kullanıcı, sistemin giriş arayüzüne kimlik numarası ve şifresini girerek süreci başlatır. Arayüz, bu bilgileri arka planda çalışan kimlik doğrulama servisine iletir. Servis, veritabanına sorgu atarak ilgili kullanıcı bilgilerini getirir ve şifrenin doğruluğunu kendi içinde kontrol eder. Eğer şifre eşleşirse (doğrulama başarılı), sistem oturumu başlatarak kullanıcıyı ana ekrana yönlendirir. Doğrulamanın başarısız olduğu alternatif durumda ise işlem durdurulur ve arayüz üzerinden kullanıcıya giriş hatası mesajı gösterilir.

Senaryo 2: Müşterinin İki Hesap Arasında Para Transferi (EFT/Havale) Yapması

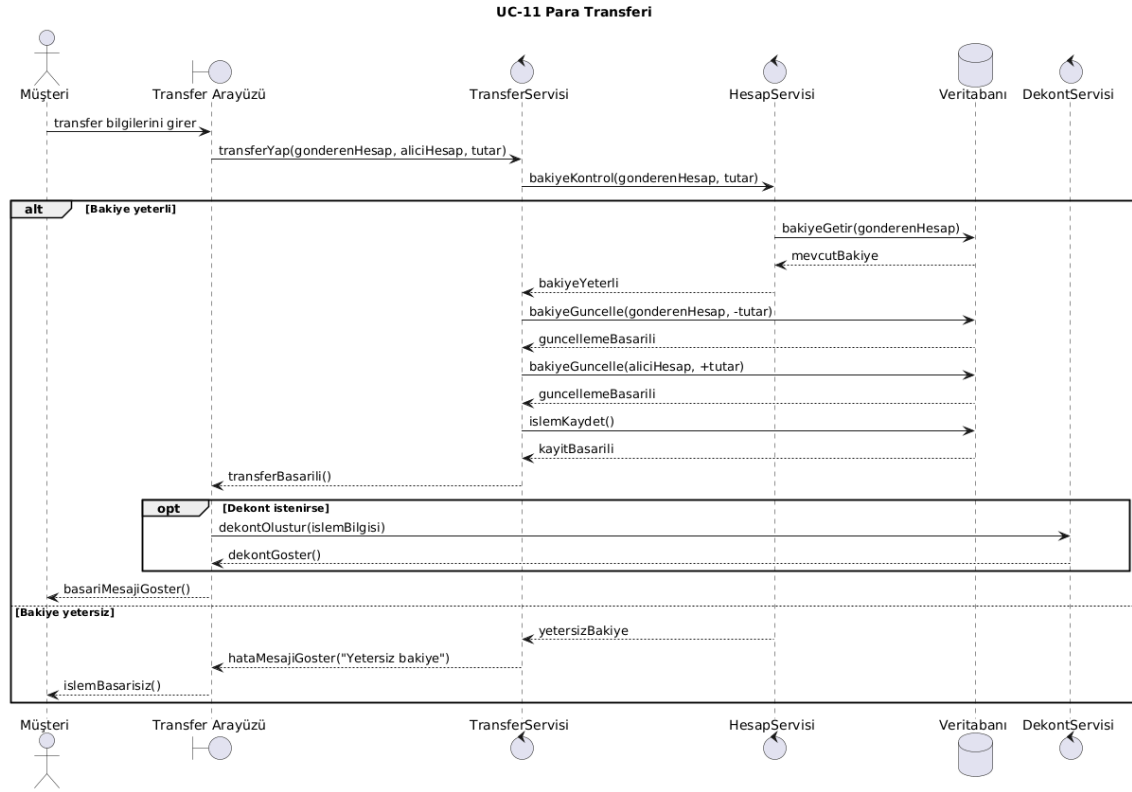


Figure 7: Müşteri Para Transferi İşlemi (UC-11) Sıra Diyagramı

Müşteri, gönderici hesap, alıcı hesap ve transfer tutarı bilgilerini arayüze girerek transfer talebi oluşturur. Transfer servisi, işlemi gerçekleştirmeden önce hesap servisi aracılığıyla gönderen hesabın bakiyesini kontrol eder. Eğer bakiye işlem için yeterliyse veritabanı ile bağlantı kurularak önce gönderen hesabın bakiyesi azaltılır, ardından alıcı hesabın bakiyesi artırılır. Bu iki güncelleme başarılı olduktan sonra işlem log olarak kaydedilir ve arayüze başarılı mesajı iletilir. Transferin tamamlanmasının ardından müşteri talep ederse dekont oluşturulup gösterilebilir. Başlangıçtaki bakiye kontrolü başarısız olursa, hiçbir veritabanı işlemi yapılmadan süreç sonlandırılır ve kullanıcıya bakiye yetersizliği hatası yansıtılır.

Senaryo 3: Müşterinin Kendi Hesabından Para Çekmesi

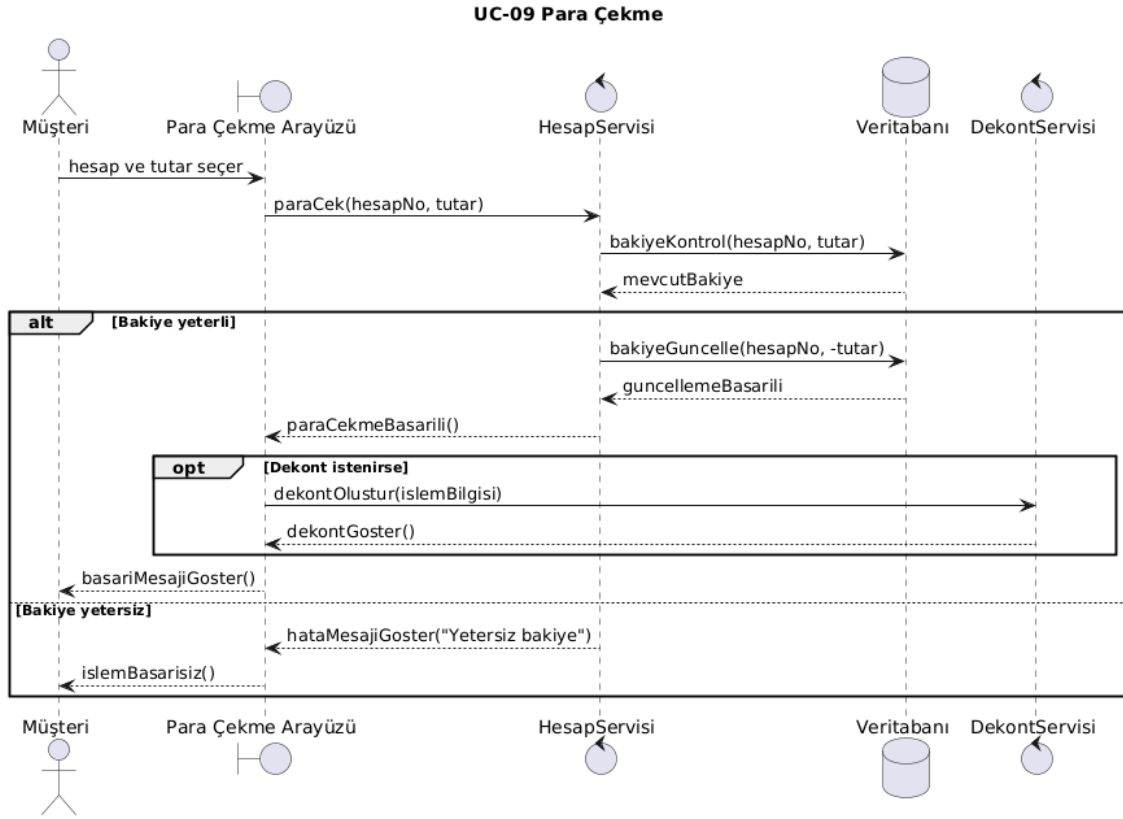
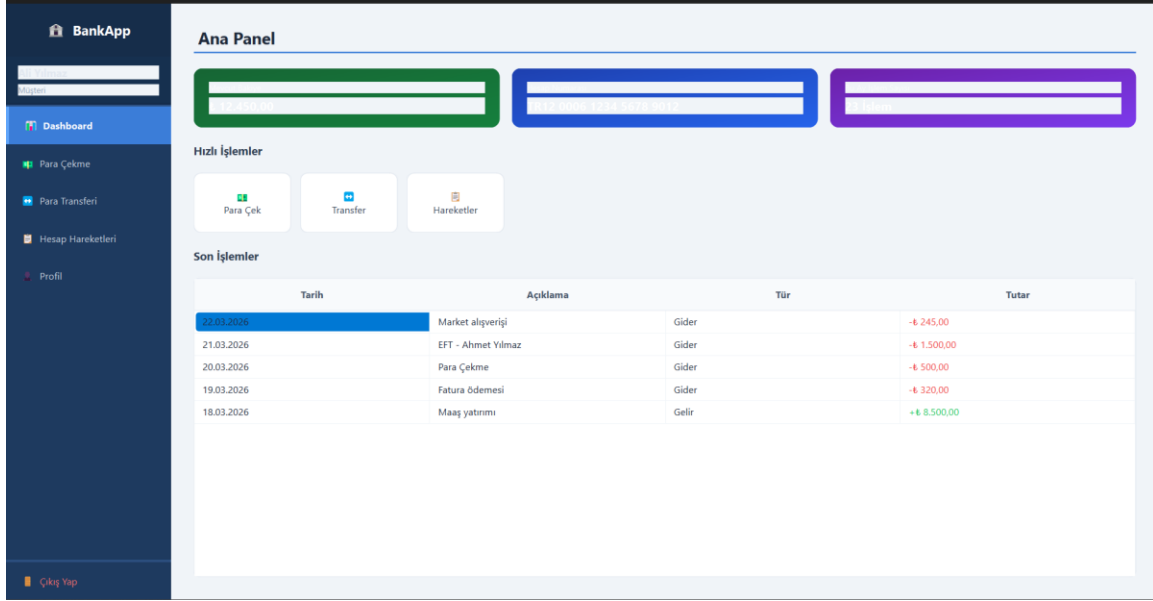


Figure 8: Müşteri Para Çekme İşlemi (UC-09) Sıra Diyagramı

Müşteri, arayüz üzerinden işlem yapmak istediği hesabını ve çekeceği tutarı seçer. Arayüz, bu talebi hesap servisine iletir ve servis kendi içerisinde müşterinin mevcut bakiyesini kontrol eder. Bakiye yeterli ise veritabanı ile iletişime geçilerek ilgili tutar hesaptan düşülür ve bakiye güncellemesi başarıyla tamamlanır. İşlem başarılı olduktan sonra müşteri dilerse (opsiyonel olarak) dekont servisi tetiklenir ve ekranda işlem dekontu gösterilir. Bakiyenin yetersiz olması durumunda ise veritabanında hiçbir değişiklik yapılmaz ve müşteriye "Yetersiz bakiye" uyarısı verilerek işlem iptal edilir.

4.6 Kullanıcı Arayüzü



5. Referanslar

https://github.com/yitepeli/CS319Project_Group2D_LOST

<https://www.cs.bilkent.edu.tr/~eraytuzun/teaching/cs319/SampleProjects/Fall2022-Sample-AnalysisReport.pdf>

<https://www.cs.bilkent.edu.tr/~eraytuzun/teaching/cs319/SampleProjects/Fall2021-Sample-AnalysisReport.pdf>

<https://github.com/kaansancak/1B.battlecity-cs319/>